



give me a comprehensive report on affine discrete-time nonlinear systems assuming no starting knowledge of discrete mathematics

Affine discrete-time nonlinear systems are mathematical models that describe how a system's state evolves in discrete steps when the dependence on the control input is "almost linear," which makes them much easier to analyze and control than fully general nonlinear systems. Even starting with no discrete-math background, they can be understood using basic algebra, functions, and vectors. ^{[1] [2] [3]}

Core idea and notation

An affine discrete-time nonlinear system has the form

$$x_{k+1} = f(x_k) + g(x_k)u_k$$

where $k = 0, 1, 2, \dots$. ^{[2] [3] [1]}

- **State x_k :** An n -dimensional vector that summarizes the system at step k (for example, position and velocity of a robot). ^{[3] [2]}
- **Input u_k :** An m -dimensional control vector chosen at step k (for example, motor voltages or forces). ^{[2] [3]}
- **Nonlinear maps f, g :**
 - $f: \mathbb{R}^n \rightarrow \mathbb{R}^n$ describes the "natural" evolution with no input.
 - $g: \mathbb{R}^n \rightarrow \mathbb{R}^{n \times m}$ scales the input and can depend nonlinearly on the state. ^{[1] [2]}

This structure is called **control-affine** (affine in u_k) because for each fixed state x_k , the right-hand side is a linear function of u_k plus an offset. ^{[4] [5]}

Affine vs linear vs general nonlinear

A few special cases show why the affine structure is important. ^{[3] [1]}

- **Linear discrete-time system:**

$$x_{k+1} = Ax_k + Bu_k$$

Here both state and input enter linearly with constant matrices A, B . ^{[6] [3]}

- **General nonlinear system:**

$$x_{k+1} = F(x_k, u_k)$$

No structural restrictions; analysis and optimal control are much harder. ^{[7] [3]}

- **Affine nonlinear system (control-affine):**

$$x_{k+1} = f(x_k) + g(x_k)u_k$$

Nonlinear in the state, but linear in u_k , which preserves many control design tools (e.g., feedback linearization, dynamic programming-based methods) while modeling rich nonlinear behavior. ^{[8] [5] [1]}

Because of this compromise, affine models are widely used for mechanical systems, robotics, and many sampled-data controllers. ^{[4] [2] [3]}

Discrete time and trajectories

Discrete time means the system is updated at isolated time instants $t_k = kT$ for some sampling period T . ^{[6] [3]}

- A **trajectory** is a sequence $x_0, x_1, x_2, \dots$ generated by repeatedly applying

$$x_{k+1} = f(x_k) + g(x_k)u_k$$

starting from an initial state x_0 . ^[3]

- Choosing different input sequences $u_0, u_1, \dots$ produces different trajectories from the same initial state. ^[3]
- In digital control, the controller runs on a computer and updates u_k at each sampling instant based on the current state (or its estimate). ^{[9] [6]}

This recursive update rule is the discrete-time counterpart of differential equations used in continuous-time nonlinear dynamics. ^[3]

Stability and equilibria

Two basic concepts for these systems are **equilibrium points** and **stability**. ^{[1] [3]}

- An **equilibrium** satisfies
so if the system ever reaches and the input is held at , it stays there forever. ^[3]
- The equilibrium is **stable** if trajectories starting "close enough" to stay close for all future steps. ^[3]
- A common way to assess stability for nonlinear discrete-time systems is to construct a Lyapunov function $V(x)$ that decreases along trajectories, generalizing energy concepts from physics. ^{[1] [3]}

For affine systems, the structure often simplifies stability proofs and controller designs, especially when combined with feedback laws $u_k = \kappa(x_k)$. ^{[10] [1]}

Control design and why the affine form helps

Control design asks: how should u_k be chosen to make the system behave in a desired way (stabilize, track a reference, minimize a cost)? ^{[8] [10] [1]}

Common themes for affine discrete-time nonlinear systems include:

- **Optimal control and dynamic programming**
 - Problems are often posed by minimizing a cost such as

$$J = \sum_{k=0}^{\infty} (x_k^\top Q x_k + u_k^\top R u_k)$$

subject to $x_{k+1} = f(x_k) + g(x_k)u_k$.^{[10] [8] [1]}

- Solving the corresponding discrete-time Hamilton–Jacobi–Bellman equation directly is difficult, so approximate dynamic programming or neural dynamic programming methods are frequently used.^{[11] [8] [1]}

- **Feedback linearization (for affine systems)**

- For a class of affine systems satisfying certain rank and relative degree conditions, one can design a feedback law $u_k = \alpha(x_k) + \beta(x_k)v_k$ that cancels nonlinearities and yields an approximately linear closed-loop mapping in terms of a new input v_k .^[5]
- This exploits the linearity in u_k ; general nonlinear dependence on u_k would make such exact cancellation fundamentally harder.^[5]

- **Data-driven and Koopman/DMD-based methods**

- Recent work represents control-affine discrete-time systems by linear operators acting on lifted functions of the state, enabling linear prediction in high-dimensional feature spaces.^[2]
- This bridges nonlinear system identification and linear model-based control using time-series data.^{[9] [2]}

Because the input appears affinely, structure-preserving numerical algorithms and learning methods (e.g., policy iteration variants, kernel-based DMD) can be specialized to this class.^{[11] [2] [1]}

Relation to discrete mathematics assumptions

Understanding these systems does not require formal discrete mathematics (proof techniques, graph theory, etc.) but does rest on some basic notions.^{[12] [3]}

- **Sequences:** The index k enumerates elements of sequences $\{x_k\}$ and $\{u_k\}$, which is enough to talk about discrete time and trajectories.^{[12] [3]}
- **Functions on vectors:** Nonlinear maps f, g just assign new vectors to old ones; familiarity with multivariable functions and matrices is sufficient.^{[13] [12]}
- **Linear algebra:** Many analyses (stability near equilibria, linearization, and operator-based methods) hinge on matrix calculus and eigenvalues rather than combinatorial discrete math.^{[7] [3]}

So the conceptual load is mainly in linear algebra, calculus, and dynamical systems rather than in discrete structures.

If desired, the next step can be a worked example: starting from a simple continuous-time nonlinear system, discretizing it, writing it in affine form, and then walking through stability and a basic feedback design.

1. https://scholarsmine.mst.edu/cgi/viewcontent.cgi?article=7069&context=ele_comeng_facwork
2. <https://arxiv.org/pdf/2309.09817.pdf>
3. <https://www.vandewouw.dc.tue.nl/Jungers2025Enceyclopedia.pdf>
4. <https://lavalle.pl/planning/node825.html>
5. <https://www3.nd.edu/~lemmon/courses/ee580/lectures/chapter9.pdf>
6. https://idsc.ethz.ch/content/dam/ethz/special-interest/mavt/dynamic-systems-n-control/idsc-dam/Lectures/Digital-Control-Systems/Slides_DigReg_2013.pdf
7. <https://grizzle.robotics.umich.edu/files/LinAlgDiscreteTimeNLsystems.pdf>
8. <https://pubmed.ncbi.nlm.nih.gov/19596551/>
9. https://python-control.readthedocs.io/en/0.10.2/examples/simulating_discrete_nonlinear.html
10. https://scholarsmine.mst.edu/cgi/viewcontent.cgi?article=7191&context=ele_comeng_facwork
11. <https://ieeexplore.ieee.org/document/5164741/>
12. https://people.math.wisc.edu/~chr/am205/slides/am205_unit4.pdf
13. <https://www.cis.upenn.edu/~cis6100/geombchap2.pdf>
14. <http://www.sontaglab.org/FTPDIR/state-affine-realiz.pdf>
15. <https://www.sciencedirect.com/science/article/abs/pii/S221065022500210X>
16. <https://arxiv.org/pdf/2510.22089.pdf>
17. https://www.jhuapl.edu/spsa/PDF-SPSA/SpallCristion_TAC98.pdf
18. https://www.dcsc.tudelft.nl/~bdeschutter/pub/rep/23_027.pdf
19. <https://www.youtube.com/watch?v=s0XxdbuQWFU>
20. <https://www.sciencedirect.com/science/article/abs/pii/S0167691198000528>